Adding a New Organization to the Chaincert Channel

Outline:

- Start the Fabric Network
- Prepare the Directories for DLSU
- Create CA Directory for DLSU
- Copy Over Configuration and Scripts
- Deploy DLSU's Certificate Authority
- Sanity-Check Your Deployment
- Register and Enroll DLSU Admin and Peer
- Inspect and Verify DLSU's Credentials and Artifacts
- Create DLSU's Organization Definition for the Channel
- Fetch and Decode Current Channel Configuration
- Merge DLSU into Channel Configuration
- Review and Prepare Your Membership Update for Signing
- Sign and Submit the Channel Update
- Switch to DLSU Admin, Start its Peer, and Join the Channel
- DLSU Peer Joins the Chaincert Channel
- Verify Channel Membership and Install Chaincode
- Test Credential Issuance as DLSU
- CREATED CONNECTION PROFILE FOR DLSU MANUALLY
- Created couch db for dlsu (inside addDLSU/compose)
- Import DLSU Admin into the Backend Walle
- Quick Reference: Switching Between Orgs

---

1. Start the Fabric Network

```
# ============================================
# COMPLETE FRESH START CLEANUP
# ============================================

# 1. Navigate to test-network
cd ~/fabric-samples/fabric-samples/test-network

TAS DO THIS PARA MAKAPAGPEER COMMANDS
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=${PWD}/../config/


# 2. Stop and remove DLSU containers and volumes
cd addDLSU/compose
docker-compose -f compose-dlsu.yaml down -v
docker-compose -f compose-ca-dlsu.yaml down -v
docker-compose -f compose-couch-dlsu.yaml down -v

# 3. Go back to test-network and clean base network
cd ~/fabric-samples/fabric-samples/test-network
./network.sh down     # Clean start
# Use sudo to force remove
sudo rm -rf organizations/fabric-ca/dlsu
sudo rm -rf organizations/peerOrganizations/dlsu.example.com
rm -rf addDLSU/DLSUMSP.json
rm -f chaincert*.tar.gz

PENDING - DELETE MGA IDS SA WALLET DIRECTORY TAS MAG NODE
WALLET.JS NALANG ULIT AFTER NETWORK UP

cd ~/fabric-samples/fabric-samples/test-network
```

```
./network.sh up createChannel -ca -s couchdb -c
chaincert-channel


#
cd ~/fabric-samples/fabric-samples
curl -sSL https://bit.ly/2ysbOFE | bash -s

export PATH="$PATH:$(pwd)/bin"
```

- HELPFUL COMMAND --> tree -I 'node_modules' (para ma navigate mo directory mo | need download tree tho search nalang command)

What's happening here?
We're ensuring a fresh network by shutting it down before starting again. This creates the `chaincert-channel` (think of it as a private group chat for all orgs), and sets up the Certificate Authority (CA) and CouchDB (the database for your blockchain data). It's like reopening your school with all new security and record-keeping systems ready for new members.

---

2. Prepare Directories for DLSU

```
Bash

In test-network

mkdir addDLSU
cd addDLSU
mkdir compose
mkdir fabric-ca
```

```
cd compose
mkdir docker
cd docker
mkdir peercfg
cd ../../
```

What's happening here?
This is the organizational phase. We're setting up directories for DLSU-related config and cert files, starting with a main folder and subfolders for each purpose—much like preparing separate envelopes and filing cabinets for new paperwork and keys.

---

3. Create CA Directory for DLSU

```
bash

cd
~/fabric-samples/fabric-samples/test-network/organizations/fabric-ca
mkdir dlsu
```

What's happening here?
We're creating a dedicated secure vault for all of DLSU's certificates and keys issued by its own CA. Think of this as prepping the building where DLSU's digital IDs will be printed and stored.

---

4. Copy Over Configuration and Scripts

```
bash

cp <your-path>/peercfg/core.yaml
~/fabric-samples/fabric-samples/test-network/addDLSU/compose/doc
ker/peercfg/core.yaml
cp <your-path>/configtx-WITHDLSU.yaml
~/fabric-samples/fabric-samples/test-network/configtx/configtx.y
aml
cp <your-path>/compose/compose-dlsu.yaml
~/fabric-samples/fabric-samples/test-network/addDLSU/compose/com
pose-dlsu.yaml
cp <your-path>/compose/compose-ca-dlsu.yaml
~/fabric-samples/fabric-samples/test-network/addDLSU/compose/com
pose-ca-dlsu.yaml
cp <your-path>/fabric-ca/registerEnrollFORDLSU.sh
~/fabric-samples/fabric-samples/test-network/addDLSU/fabric-ca/r
egisterEnroll.sh
```

PURPOSE OF THESE FILES COPIED TO UBUNTU:

cp <your-path>/peercfg/core.yaml → addDLSU/compose/docker/peercfg/core.yaml

Use:

This is the main configuration file for a Fabric peer node. It tells the peer how to behave—storage settings, logging, MSP, TLS, networking, and more.

Why copy?

We need DLSU's peer to follow the network's standards and security rules. Think of this like copying the building's "operations manual" so the new DLSU branch knows all the procedures and settings needed to be compatible with your campus or office.

---

cp <your-path>/configtx-WITHDLSU.yaml → configtx/configtx.yaml

Use:
This file defines the overall structure and participants for channels (including DLSU as a new member), consensus policies, and what “profiles” (channel templates) exist.

Why copy?
We replace the old with one that now includes DLSU’s organization details. Think of this like updating the school’s official list of clubs so that DLSU appears as a registered organization allowed to join meetings and events.

---

cp <your-path>/compose/compose-dlsu.yaml → addDLSU/compose/compose-dlsu.yaml

Use:
This Docker Compose file describes how to run DLSU’s own peer node—what image, environment variables, ports, and volumes to use.

Why copy?
You need it to tell Docker exactly how to construct (instantiate) DLSU’s peer as a running container in the network. Like giving construction blueprints to your contractor for building a new DLSU satellite office in your town.

---

cp <your-path>/compose/compose-ca-dlsu.yaml → addDLSU/compose/compose-ca-dlsu.yaml

Use:
This Docker Compose file describes how to launch DLSU’s Certificate Authority as a container.

Why copy?

You need a way to generate/verify DLSU's identities and certs. This is like providing the "printer and ID office instructions" so DLSU can make official badges/IDs for its staff.

---

cp <your-path>/fabric-ca/registerEnrollFORDLSU.sh → addDLSU/fabric-ca/registerEnroll.sh

Use:

This script automates enrollment and registration of DLSU's peer and admin users with the CA, generating all their needed certificates and private keys.

Why copy?

It saves you from manually typing complex commands for every ID request. Like giving an HR team a script to mass-produce ID cards for all DLSU employees in one go—streamlining onboarding without typos or errors.

What's happening here?

We're equipping DLSU with the blueprint files it needs to operate — Docker configuration for its peer and CA, network setup configs, and custom enrollment scripts. It's like importing the forms and checklists from another branch with proven workflows.

```
kaitanz_@KingKaiLappy:~/fabric-samples/fabric-samples/test-network/addDLSU$ tree
.
├── DLSUMSP.json
├── ccp-generate.sh
├── ccp-template.json
├── ccp-template.yaml
├── compose
│   ├── compose-ca-dlsu.yaml
│   ├── compose-couch-dlsu.yaml
│   ├── compose-dlsu.yaml
│   ├── docker
│   │   └── peercfg
│   │       └── core.yaml
│   └── peercfg
├── dlsu-crypto.yaml
└── fabric-ca
    └── registerEnroll.sh

6 directories, 10 files
kaitanz_@KingKaiLappy:~/fabric-samples/fabric-samples/test-network/addDLSU$
```

Example output:

---

5. Deploy DLSU's Certificate Authority

bash

```
cd ~/fabric-samples/fabric-samples/test-network/addDLSU
docker-compose -f compose/compose-ca-dlsu.yaml up -d ca_dlsu
```

What's happening here?
We're spinning up DLSU's own CA in the background. The CA manages secure identities for DLSU in the network. Think of this as hiring and activating the digital ID manager for all your incoming DLSU members.

---

6. Sanity-Check Your Deployment

bash

```
curl http://localhost:17154/healthz
docker ps
```

What's happening here?
First, the curl command "knocks on the door" of your new CA to ask if it's running and healthy. The docker ps command shows all running containers, letting you confirm visually that everything is up. This is your digital security guard waving at you from reception.

---

7. Register and Enroll DLSU User and Peer

bash

```
cd
~/fabric-samples/fabric-samples/test-network/addDLSU/fabric-ca
chmod +x registerEnroll.sh
source registerEnroll.sh createDLSU
```

What's happening here?
We're running a script to ask the newly started CA to issue digital certificates for the DLSU administrator and peer node. This is like sending your HR team to collect their new company IDs and keys right after hiring.

---

8. Inspect and Verify DLSU Artifacts (test-network directory)

Check Organization Folder and MSP Layout

```
ls -l organizations/peerOrganizations/dlsu.example.com/
ls -l
organizations/peerOrganizations/dlsu.example.com/peers/peer0.dls
u.example.com/msp
ls -l
organizations/peerOrganizations/dlsu.example.com/users/Admin@dls
u.example.com/msp
```

Inspect Actual Certs and Keys

```
cat
organizations/peerOrganizations/dlsu.example.com/peers/peer0.dls
u.example.com/msp/signcerts/cert.pem | head
cat
organizations/peerOrganizations/dlsu.example.com/users/Admin@dls
u.example.com/msp/signcerts/cert.pem | head
```

Verify Private Keys

```
ls
organizations/peerOrganizations/dlsu.example.com/peers/peer0.dls
u.example.com/msp/keystore
ls
organizations/peerOrganizations/dlsu.example.com/users/Admin@dls
u.example.com/msp/keystore
```

Check NodeOUs and config.yaml

```
cat
organizations/peerOrganizations/dlsu.example.com/msp/config.yaml
```

```
cat
organizations/peerOrganizations/dlsu.example.com/peers/peer0.dls
u.example.com/msp/config.yaml
cat
organizations/peerOrganizations/dlsu.example.com/users/Admin@dls
u.example.com/msp/config.yaml
```

TLS Certs for Secure Comms

```
ls
organizations/peerOrganizations/dlsu.example.com/peers/peer0.dls
u.example.com/tls/
cat
organizations/peerOrganizations/dlsu.example.com/peers/peer0.dls
u.example.com/tls/server.crt | head
cat
organizations/peerOrganizations/dlsu.example.com/peers/peer0.dls
u.example.com/tls/server.key | head
```

Org-Level CA Certificates

```
ls organizations/peerOrganizations/dlsu.example.com/ca/
ls organizations/peerOrganizations/dlsu.example.com/tlsca/
```

What's happening here?

We’re confirming that DLSU’s folders, certificates, private keys, and security config files are present. Just like checking each recipient got their ID, password, and locker key after orientation, we want to be sure everything was issued and stored correctly.

---

9. Create DLSU’s Organization Definition for the Channel

Bash

configtx-WITHDLSU.yaml

cp "/mnt/c/Users/kaita/OneDrive/Desktop/CAP-2459-IT-AFTER-RESET/backend/node_creation/dlsu/configtx-WITHDLSU.yaml" ~/fabric-samples/fabric-samples/test-network/configtx/configtx.yaml

```
cd ~/fabric-samples/fabric-samples/test-network/configtx
export FABRIC_CFG_PATH=$PWD
```

!! Added DLSU sa configtx.yaml file under test-network/configtx/configtx.yaml (sa organizations and profile may binago, no need to do anything since u already copied the said file earlier from windows to ubuntu, yung cp commands na ginawa kanina)

```
configtxgen -printOrg DLSUMSP > ../addDLSU/DLSUMSP.json
```

What's happening here?
Here, you’re asking Fabric’s config tools to "write a summary doc" of DLSU based on its configuration. This JSON is the official statement of DLSU’s identity and permissions that will be merged into the network channel’s membership directory.

---

10. Fetch and Decode Current Channel Configuration

bash

```
cd ~/fabric-samples/fabric-samples/test-network

export FABRIC_CFG_PATH=$PWD/../config
```

```
export
ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com
/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-c
ert.pem

export CORE_PEER_TLS_ENABLED=true

export CORE_PEER_LOCALMSPID="Org1MSP"

export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizatio
ns/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/o
rg1.example.com/users/Admin@org1.example.com/msp

export CORE_PEER_ADDRESS=localhost:7051

peer channel fetch config config_block.pb -o localhost:7050
--ordererTLSHostnameOverride orderer.example.com -c
chaincert-channel --tls --cafile
${PWD}/organizations/ordererOrganizations/example.com/orderers/o
rderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

Decode the Config Block to JSON | Let's extract the channel configuration in a format you can edit (JSON):

```
configtxlator proto_decode --input config_block.pb --type
common.Block --output config_block.json

jq .data.data[0].payload.data.config config_block.json >
config.json
```

What's happening here?
We're "logging in" as Org1 admin para makapag fetch ng latest rulebook for the chaincert channel, and decoding it so we can make edits. This is like getting the house deed and translating it to your language so you can add your own name as a co-owner.

---

11. Merge DLSU into Channel Configuration

```
bash

jq -s '.[0] *
{"channel_group":{"groups":{"Application":{"groups":{"DLSUMSP":
.[1]}}}}}' config.json addDLSU/DLSUMSP.json >
modified_config.json
```

Encode Back to Protobuf | Convert both the original and updated config files to protobuf format—this is required for Fabric channel updates.

```
configtxlator proto_encode --input config.json --type
common.Config --output original_config.pb

configtxlator proto_encode --input modified_config.json --type
common.Config --output modified_config.pb
```

Compute the Channel Config Update | Generate the delta (change update) between your original and modified configs, so the channel knows what's new and only the difference is applied.

```
configtxlator compute_update --channel_id chaincert-channel
--original original_config.pb --updated modified_config.pb
--output config_update.pb
```

What's happening here?
Here's where the new org gets "written into the book": you merge DLSU's definition into the current channel config, encode both into binary, and create a "difference file" that only contains the new info you're adding. This file is your request for membership, ready for signatures.

---

12. Review and Prepare Your Membership Update for Signing | Decode the delta (for inspection and wrapping):

```
Bash
```

Decode the delta (for inspection and wrapping):

```
configtxlator proto_decode --input config_update.pb --type
common.ConfigUpdate > config_update.json
```

Wrap the Delta in an Envelope (Required for Channel Update)

para you're ready to build the envelope JSON object, which packages your config_update.json for endorsement by orgs and submission to the orderer. This builds config_update_in_envelope.json with the correct structure expected by Fabric.

```
jq -n '{payload: {header: {channel_header: {channel_id:
"chaincert-channel", type: 2}}, data: {config_update:
(input)}}}' config_update.json > config_update_in_envelope.json
```

Once you have config_update_in_envelope.json, you can encode with configtxlator:

```
configtxlator proto_encode --input
config_update_in_envelope.json --type common.Envelope --output
config_update_in_envelope.pb
```

What's happening here?
We're taking a final look at the update (proofreading), then wrapping it in a special "envelope" so it can be signed and submitted. This envelope collects the approval signatures needed for the update to be accepted.

---

13. Sign and Submit the Channel Update / Collect Signatures and Update Channel)

Pirma ka as Org1

```
peer channel signconfigtx -f config_update_in_envelope.pb

# Now switch to Org2 admin and sign
export CORE_PEER_LOCALMSPID="Org2MSP"
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/o
rg2.example.com/users/Admin@org2.example.com/msp
```

```
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizatio
ns/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
export CORE_PEER_ADDRESS=localhost:9051
peer channel signconfigtx -f config_update_in_envelope.pb

# Switch back to Org1
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/o
rg1.example.com/users/Admin@org1.example.com/msp
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizatio
ns/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_ADDRESS=localhost:7051



# Update the channel with latest config of DLSU being part of
the channel
peer channel update -f config_update_in_envelope.pb -c
chaincert-channel -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile
${PWD}/organizations/ordererOrganizations/example.com/orderers/o
rderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

# Verify
```

```
peer channel getinfo -c chaincert-channel
```

What's happening here?
Both Org1 and Org2 sign off the membership update (as if two board members are co-signing a new partnership contract). The update is then submitted to the orderer, who approves the change and adds DLSU as an official member—your network's "membership book" just got a new entry!

---

14. Switch to DLSU Admin, Start its Peer, and Join the Channel

Switch to DLSU admin

```
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="DLSUMSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizatio
ns/dlsu.example.com/peers/peer0.dlsu.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/d
lsu.example.com/users/Admin@dlsu.example.com/msp
export CORE_PEER_ADDRESS=localhost:12051
```

Start dlsu peer container

```
cd ~/fabric-samples/fabric-samples/test-network/addDLSU/compose
docker-compose -f compose-dlsu.yaml up -d
```

Verify

```
docker ps | grep peer0.dlsu
docker exec peer0.dlsu.example.com peer channel list
```

What's happening here?
We switch our CLI "identity" to DLSU—setting all the right environment variables. We then start the DLSU peer node (its worker computer) as a container. Checking status, we make sure it’s up and waiting to be let in to the channel!

---

15. DLSU Peer Joins Chaincert Channel

Fetch the Channel Block**

TEST NETWORK DAPAT

```
peer channel fetch 0 channel-artifacts/chaincert-channel.block
-o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com -c chaincert-channel --tls --cafile
"${PWD}/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem"
```

JOIN THE PEER TO CHAINCERT

```
docker exec \
  -e CORE_PEER_TLS_ENABLED=true \
  -e CORE_PEER_LOCALMSPID="DLSUMSP" \
  -e
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledg
```

```
er/fabric/peer/organizations/peerOrganizations/dlsu.example.com/peers/peer0.dlsu.example.com/tls/ca.crt \

  -e CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/organizations/peerOrganizations/dlsu.example.com/users/Admin@dlsu.example.com/msp \

  -e CORE_PEER_ADDRESS=localhost:12051 \

  peer0.dlsu.example.com \

  peer channel join -b channel-artifacts/chaincert-channel.block
```

What's happening here?
We fetch the genesis block (the channel's very first block, containing membership and initial settings), then run a command INSIDE the DLSU peer container so it can officially join the channel. Think of this as DLSU getting its office key, access card, and logging into the company network for the first time. If this joins successfully, DLSU starts syncing and participating in all future channel activities!

---

16. Verify Channel Membership and Install Chaincode

```
# Verify if part na sa peer ng chaincert-channel

docker exec peer0.dlsu.example.com peer channel list
```

as DLSU admin (make sure nagexport ka nung previous commands, install the chaincode kay dlsu peer

NEED DEPLOY CHAINCODE MUNA SA NETWORK PARA MAGKARON NG TAR.GZ FILE

```
./network.sh deployCC \
 -c chaincert-channel \
 -ccn chaincert \
 -ccl javascript \
 -ccp ~/chaincert-chaincode \
 -ccv 1.0  # nov 26 1209am
```

Verify

```
peer lifecycle chaincode querycommitted --channelID
chaincert-channel --name chaincert

peer lifecycle chaincode install chaincert.tar.gz
```

Verify

```
peer lifecycle chaincode queryinstalled
```

Verify

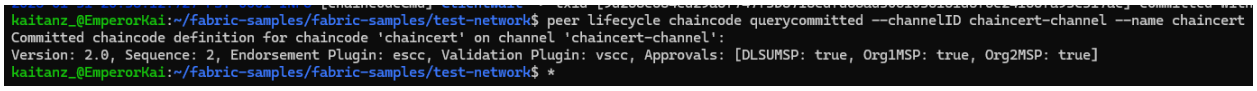
```
peer lifecycle chaincode querycommitted --channelID
chaincert-channel --name chaincert
```

As DLSU Admin, approve the chaincode na isasalpak sayo

```
# Approve with FULL package ID (with prefix)
```

```
peer lifecycle chaincode approveformyorg -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --channelID chaincert-channel \
  --name chaincert \
  --version 1.0 \
  --package-id
chaincert_1.0:1933927ebeca046496a17254a40fb15b58a36e09238099b40c
49138d83232a19 \
  --sequence 1 \
  --tls \
  --cafile
"${PWD}/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem"

# Verify
peer lifecycle chaincode querycommitted --channelID
chaincert-channel --name chaincert
```



What's happening here?
First, we double-check that DLSU's peer is really part of the channel—like confirming its name in the attendance list! Next, we install the chaincode (business logic) so DLSU can participate in credential issuance. Approving and committing the chaincode is like voting to activate a new business process for the whole team.

---

19) Created couch db for dlsu (inside addDLSU/compose)

```
docker-compose -f compose-couch-dlsu.yaml up -d couchdb_dlsu
```

Then Added

- Wallet_issuer_credentials
- Issued_credentials

Database sa couchdb ng dlsu manually via the site itself

17. Test Credential Issuance as DLSU

```
bash

peer chaincode invoke -o localhost:7050 \

  --ordererTLSHostnameOverride orderer.example.com \

  --tls \

  --cafile
"${PWD}/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" \

  -C chaincert-channel \

  -n chaincert \

  -c '{"Args":["CreateCredentialTemplate", "cred001",
"issuer_id_here", "Diploma", "Bachelor Diploma", "Bachelor
Degree in IT", "{\"course\": \"BSIT\", \"year\": 2024}"]}'
```

What's happening here?
The final moment: DLSU tries to issue a real credential! If this command works, DLSU is now a functioning, trusted member of the blockchain network, able to issue credentials that can be verified across all organizations. It’s like an employee successfully making their first transaction on their first day at work.

18) CREATED CONNECTION PROFILE FOR DLSU MANUALLY

- Need to kasi this is used to link the fabric identity of the issuer account sa peer node account
- And edited such that org1 always approves of all transactions done by dlsu to satisfy the majority endorsement policy, otherwise the transacations like issuing credentials wont push through, need pa mano mano pirmahan ng mga orgs

20. Import DLSU Admin into the Backend Wallet
(in Ubuntu)

```
cd ~/chaincert-api

node setup_Wallet_DLSU.js

And do the other
```

What's happening here?
This copies `Admin@dlsu.example.com`'s cert and key into `chaincert-api/wallet/delasalleuniversity-admin.id`, so the backend can act as DLSU when issuing and verifying credentials.

Reason: by this point, sure na na-create na yung MSP folders at DLSU admin certs, kaya guaranteed na magwo-work `setup_Wallet_DLSU.js`, and malinaw sa groupmates na last step ito after fabric-side DLSU setup.

QUICK REFERENCE PARA MAKAPAGSWITCH NG ACCOUNTS/ORGS AGAD (in test-network directory)

DLSU

export CORE_PEER_TLS_ENABLED=true

export CORE_PEER_LOCALMSPID="DLSUMSP"

export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/dlsu.example.com/peers/peer0.dlsu.example.com/tls/ca.crt

export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/dlsu.example.com/users/Admin@dlsu.example.com/msp

export CORE_PEER_ADDRESS=localhost:12051

Org1

export CORE_PEER_TLS_ENABLED=true

export CORE_PEER_LOCALMSPID="Org1MSP"

export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp

export CORE_PEER_ADDRESS=localhost:7051

Org2

bash

export CORE_PEER_TLS_ENABLED=true

export CORE_PEER_LOCALMSPID="Org2MSP"

export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt

export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org2.example.com/users/Admin@org2.example.com/msp

export CORE_PEER_ADDRESS=localhost:9051